Mastering Hibernate ORM & Spring Data JPA -- Internals, Architecture, and Enterprise Usage Guide

---

Author: Senior Java Architect **Target Audience:** Java developers with 3+ years of experience aiming for senior/architect-level roles **Prerequisites:** Core Java, Spring Boot, SQL fundamentals, JDBC basics

---

Table of Contents

1. Introduction to ORM
2. What is Hibernate
3. Hibernate Core Architecture
4. Entity Mapping
5. Hibernate Entity Lifecycle
6. Associations in Hibernate
7. Fetching Strategies
8. Hibernate Query Methods
9. Transaction Management
10. Spring Data JPA Introduction
11. Spring Data JPA Architecture
12. Spring Data JPA Repositories
13. Query Methods in Spring Data JPA
14. Custom Queries
15. Pagination and Sorting
16. Spring Data JPA Specifications
17. Performance Optimization
18. Hibernate Caching
19. Common Performance Problems
20. Real Production Scenario
21. Microservices Data Strategy
22. Design Patterns Used
23. Best Practices
24. Interview Questions (100+)
25. Coding Exercises (20)

Part 1: Introduction to ORM

1.1 What is ORM (Object Relational Mapping)?

ORM is a programming technique that maps objects in an object-oriented language (Java) to rows in a relational database. Instead of writing raw SQL and manually mapping ResultSets to Java objects, ORM frameworks handle this automatically.

```
Without ORM (Raw JDBC):
Java Object  -->  Manual SQL  -->  ResultSet  -->  Manual Mapping  -->  Java
Object

With ORM (Hibernate):
Java Object  <-->  ORM Framework  <-->  Database
              (automatic mapping)
```

Manual JDBC Example (What ORM Eliminates)

```java
// WITHOUT ORM -- Manual, error-prone, tedious
public User findById(Long id) {
    Connection conn = dataSource.getConnection();
    PreparedStatement ps = conn.prepareStatement(
        "SELECT id, name, email, created_at FROM users WHERE id = ?");
    ps.setLong(1, id);
    ResultSet rs = ps.executeQuery();

    User user = null;
    if (rs.next()) {
        user = new User();
        user.setId(rs.getLong("id"));
        user.setName(rs.getString("name"));
        user.setEmail(rs.getString("email"));
        user.setCreatedAt(rs.getTimestamp("created_at").toLocalDateTime());
    }
    rs.close(); ps.close(); conn.close();
    return user;
}

// WITH ORM (Hibernate/JPA) -- Clean, type-safe, automatic
public User findById(Long id) {
```

```
    return entityManager.find(User.class, id); // That's it!
}
```

1.2 Object Model vs Relational Model

Aspect	Object Model (Java)	Relational Model (SQL)
Structure	Classes with fields and methods	Tables with columns
Identity	Object reference (`==`, `.equals()`)	Primary key
Relationships	References (composition, inheritance)	Foreign keys, join tables
Inheritance	Class hierarchy (extends)	No native support
Encapsulation	Private fields + methods	All columns exposed
Navigation	`user.getOrders().get(0).getItems()`	JOIN queries
Collections	`List<Order>`, `Set<Item>`	Separate tables

1.3 Object-Relational Impedance Mismatch

The fundamental conflict between how data is represented in objects vs tables:

Mismatch Area	Problem	ORM Solution
Granularity	Object can have fine-grained types (Address, Money)	`@Embeddable` maps value objects
Inheritance	Java has class hierarchies; SQL has no inheritance	`@Inheritance` strategies (SINGLE_TABLE, JOINED, TABLE_PER_CLASS)
Identity	Java: reference equality vs `.equals()`	`@Id` maps to primary key
Associations	Java: object references; SQL: foreign keys	`@ManyToOne`, `@OneToMany` map relationships
Data Navigation	Java: traverse object graph; SQL: explicit JOINs	Lazy loading, fetch strategies

1.4 Problems ORM Solves

Data Mapping

```java
// ORM maps this automatically:
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;                    // -> id BIGINT PRIMARY KEY

    @Column(nullable = false)
    private String name;                // -> name VARCHAR(255) NOT NULL

    @Column(unique = true)
    private String email;               // -> email VARCHAR(255) UNIQUE

    @Enumerated(EnumType.STRING)
    private UserStatus status;          // -> status VARCHAR(255)

    private LocalDateTime createdAt;    // -> created_at TIMESTAMP
}
```

Identity Management

```java
// ORM guarantees: same DB row = same Java object within a session
User u1 = entityManager.find(User.class, 1L);
User u2 = entityManager.find(User.class, 1L);
assert u1 == u2; // TRUE -- same object reference (1st level cache)
```

Transaction Handling

```java
// ORM + Spring manages transactions declaratively
@Transactional
public void transferFunds(Long fromId, Long toId, BigDecimal amount) {
    Account from = accountRepository.findById(fromId).orElseThrow();
    Account to = accountRepository.findById(toId).orElseThrow();
    from.debit(amount);
    to.credit(amount);
    // Auto-committed on success, auto-rolled back on exception
}
```

Part 2: What is Hibernate

2.1 What is Hibernate?

Hibernate is the most popular ORM framework for Java. It implements the **JPA (Java Persistence API)** specification and provides additional features beyond the standard.

Aspect	Details
Type	ORM framework
Implements	JPA 3.1 (Jakarta Persistence)
Created by	Gavin King (2001)
Current version	Hibernate 6.x
License	LGPL (open source)

Why Hibernate Became Popular

1. **Eliminates boilerplate** -- No manual SQL, ResultSet mapping, connection management
2. **Database portability** -- Switch from MySQL to PostgreSQL without changing code
3. **Automatic schema generation** -- Create/update tables from entity classes
4. **Caching** -- Built-in first-level cache, pluggable second-level cache
5. **Lazy loading** -- Load associated data only when accessed
6. **Dirty checking** -- Automatically detects and persists state changes

2.2 How Hibernate Works Internally

```
+---------------------------------------------+
|           Java Application                  |
|  (Entities, Services, Repositories)         |
+---------------------------------------------+
         |  persist()  find()  merge()
         v
+---------------------------------------------+
|         Hibernate ORM Layer                 |
|  +------------+  +-------------+            |
|  | Session    |  | Transaction |            |
|  | Factory    |  | Manager     |            |
|  +------------+  +-------------+            |
```

```
|  +------------+  +-------------+            |
|  | 1st Level  |  | Query       |            |
|  | Cache      |  | Translator  |            |
|  +------------+  +-------------+            |
|  +------------+                             |
|  | Dialect    | (MySQL, PostgreSQL, Oracle)   |
|  +------------+                             |
+---------------------------------------------+
         |  SQL statements
         v
+---------------------------------------------+
|              JDBC Driver                    |
+---------------------------------------------+
         |
         v
+---------------------------------------------+
|        Relational Database                  |
|   (MySQL, PostgreSQL, Oracle, H2)           |
+---------------------------------------------+
```

Internal Flow: Saving an Entity

```
1. Application calls entityManager.persist(user)
2. Hibernate assigns ID (via strategy)
3. Entity added to Persistence Context (1st level cache)
4. Entity marked as "dirty" (new)
5. On transaction commit / flush:
   a. Hibernate generates INSERT SQL using Dialect
   b. SQL sent to JDBC driver
   c. JDBC executes against database
   d. Entity state changed to "persistent"
```

Part 3: Hibernate Core Architecture

3.1 SessionFactory

What: A heavyweight, thread-safe, immutable factory for creating Sessions. Built once at application startup from configuration.

Why: Creating database connections is expensive. SessionFactory pre-configures mappings, caches, and connection pools.

When: One per database per application (singleton).

```
// Spring Boot auto-configures this -- you rarely create it manually
// But understanding it is critical for interviews

// Manual creation (non-Spring):
Configuration config = new Configuration()
    .configure("hibernate.cfg.xml")
    .addAnnotatedClass(User.class);
SessionFactory factory = config.buildSessionFactory();

// In Spring Boot: auto-configured as EntityManagerFactory
@Autowired
private EntityManagerFactory entityManagerFactory;
// EntityManagerFactory wraps SessionFactory (JPA standard interface)
```

3.2 Session (EntityManager)

What: A lightweight, short-lived, non-thread-safe object representing a unit of work. Wraps a JDBC connection. **JPA equivalent: EntityManager.**

Why: Provides the API for CRUD operations and serves as the first-level cache (persistence context).

When: One per request/transaction (request-scoped in web applications).

```
// Session operations
Session session = sessionFactory.openSession();

// JPA equivalent (what Spring uses)
EntityManager em = entityManagerFactory.createEntityManager();
```

```
// CRUD operations
em.persist(user);              // INSERT
User u = em.find(User.class, 1L);  // SELECT by ID
em.merge(user);                // UPDATE
em.remove(user);               // DELETE
```

3.3 Transaction

What: Wraps a database transaction. Ensures ACID properties.

```
// Manual transaction (non-Spring)
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
try {
    session.persist(user);
    tx.commit();
} catch (Exception e) {
    tx.rollback();
    throw e;
} finally {
    session.close();
}

// Spring manages this with @Transactional -- much cleaner!
@Transactional
public void createUser(User user) {
    entityManager.persist(user); // Auto-commit or rollback
}
```

3.4 Hibernate Dialect

What: Translates HQL/JPQL to database-specific SQL.

Database	Dialect Class
MySQL 8	`org.hibernate.dialect.MySQLDialect`
PostgreSQL	`org.hibernate.dialect.PostgreSQLDialect`
Oracle	`org.hibernate.dialect.OracleDialect`

H2	`org.hibernate.dialect.H2Dialect`
SQL Server	`org.hibernate.dialect.SQLServerDialect`

```yaml
# application.yml
spring:
  jpa:
    database-platform: org.hibernate.dialect.PostgreSQLDialect
    properties:
      hibernate:
        format_sql: true
        show_sql: true
```

Part 4: Entity Mapping

4.1 Basic Annotations

@Entity

Marks a Java class as a persistent entity mapped to a database table.

```
@Entity // Required -- tells Hibernate "this is a database table"
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name; // Maps to column "name" by default
}
```

@Table

Customizes the table name and constraints.

```
@Entity
@Table(
    name = "products",
    schema = "ecommerce",
    uniqueConstraints = @UniqueConstraint(columnNames = {"sku", "region"}),
    indexes = @Index(name = "idx_product_name", columnList = "name")
)
public class Product { ... }
```

@Id and @GeneratedValue

Strategy	Description	Database	Use When
`IDENTITY`	Auto-increment column	MySQL, PostgreSQL	Single DB, simple apps
`SEQUENCE`	Database sequence	PostgreSQL, Oracle	High-performance batch inserts
`TABLE`	Uses a separate table	All	Portable but slow

`UUID`	UUID generation	All	Distributed systems
`AUTO`	Hibernate picks best	Varies	Default (not recommended)

```
// IDENTITY (MySQL, PostgreSQL auto-increment)
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

// SEQUENCE (PostgreSQL, Oracle -- preferred for performance)
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE,
    generator = "product_seq")
@SequenceGenerator(name = "product_seq",
    sequenceName = "product_sequence",
    allocationSize = 50) // Pre-allocate 50 IDs -- huge performance gain
private Long id;

// UUID (distributed systems)
@Id
@GeneratedValue(strategy = GenerationType.UUID)
private UUID id;
```

@Column

```
@Column(
    name = "product_name",    // Column name (default: field name)
    nullable = false,         // NOT NULL constraint
    unique = true,            // UNIQUE constraint
    length = 150,             // VARCHAR length (default 255)
    precision = 10,           // Decimal precision
    scale = 2,                // Decimal scale
    columnDefinition = "TEXT" // Raw SQL column definition
)
private String name;
```

4.2 Composite Keys

```java
// Method 1: @EmbeddedId
@Embeddable
public class OrderItemId implements Serializable {
    private Long orderId;
    private Long productId;

    // Constructors, equals(), hashCode() -- REQUIRED
}

@Entity
public class OrderItem {
    @EmbeddedId
    private OrderItemId id;

    private int quantity;
    private BigDecimal price;
}

// Method 2: @IdClass
@Entity
@IdClass(OrderItemId.class)
public class OrderItem {
    @Id private Long orderId;
    @Id private Long productId;
    private int quantity;
}
```

4.3 Embedded Objects (Value Objects)

```java
@Embeddable
public class Address {
    private String street;
    private String city;
    private String state;

    @Column(length = 10)
    private String zipCode;
}

@Entity
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
    private String name;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street",
            column = @Column(name = "billing_street")),
        @AttributeOverride(name = "city",
            column = @Column(name = "billing_city"))
    })
    private Address billingAddress;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street",
            column = @Column(name = "shipping_street")),
        @AttributeOverride(name = "city",
            column = @Column(name = "shipping_city"))
    })
    private Address shippingAddress;
}
// Result: ONE table "customer" with columns:
// id, name, billing_street, billing_city, ..., shipping_street, shipping_city
```

4.4 Enums and Temporal Types

```
@Entity
public class Employee {

    @Enumerated(EnumType.STRING) // Store "ACTIVE", not 0
    private EmployeeStatus status;

    @Column(updatable = false)
    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = createdAt;
    }

    @PreUpdate
```

```
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

Part 5: Hibernate Entity Lifecycle

5.1 Entity States

```
                new Entity()
                     |
                     v
            +-------------+
            |  TRANSIENT  |   Not managed, no ID, not in DB
            +-------------+
                     |
             persist()
                     |
                     v
            +-------------+
            | PERSISTENT  |   Managed by Session, has ID, in DB
            +-------------+   Changes auto-detected (dirty checking)
              |         |
      detach()/close()   remove()
              |         |
              v         v
        +----------+  +---------+
        | DETACHED |  | REMOVED |
        +----------+  +---------+
             |              Not managed, scheduled for DELETE
        merge()
             |
             v
        +-------------+
        | PERSISTENT  |   Re-attached to new Session
        +-------------+
```

5.2 State Transitions with Code

```
// TRANSIENT -> New object, not yet associated with Session
User user = new User("John", "john@email.com");
// user is TRANSIENT -- Hibernate doesn't know about it

// TRANSIENT -> PERSISTENT
```

```
entityManager.persist(user);
// user is now PERSISTENT -- managed by the persistence context
// Hibernate tracks all changes to this object

// PERSISTENT -- Dirty Checking
user.setEmail("john.updated@email.com");
// NO explicit save/update needed!
// Hibernate detects the change and generates UPDATE on flush/commit

// PERSISTENT -> DETACHED
entityManager.detach(user);
// or: entityManager.close()
// user is DETACHED -- no longer managed
user.setName("Jane"); // This change will NOT be auto-persisted

// DETACHED -> PERSISTENT (re-attach)
User managed = entityManager.merge(user);
// 'managed' is PERSISTENT; 'user' is still DETACHED
// Use the returned reference!

// PERSISTENT -> REMOVED
entityManager.remove(managed);
// Entity scheduled for DELETE on flush/commit
```

5.3 Important Details

State	In Session?	Has ID?	In DB?	Changes Tracked?
Transient	No	No	No	No
Persistent	Yes	Yes	Yes (or pending INSERT)	Yes (dirty checking)
Detached	No	Yes	Yes	No
Removed	Yes	Yes	Yes (pending DELETE)	No

Part 6: Associations in Hibernate

6.1 @ManyToOne (Most Common)

```java
@Entity
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY) // LAZY is best practice!
    @JoinColumn(name = "customer_id", nullable = false)
    private Customer customer;

    private BigDecimal totalAmount;
}
// DB: orders table has customer_id FK column
```

6.2 @OneToMany (Inverse Side)

```java
@Entity
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "customer", // Inverse side -- no FK column here
               cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Order> orders = new ArrayList<>();

    // Helper methods to maintain bidirectional consistency
    public void addOrder(Order order) {
        orders.add(order);
        order.setCustomer(this);
    }

    public void removeOrder(Order order) {
        orders.remove(order);
        order.setCustomer(null);
```

```
    }
}
```

6.3 @OneToOne

```
@Entity
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "profile_id", unique = true)
    private UserProfile profile;
}

@Entity
public class UserProfile {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(mappedBy = "profile")
    private User user;

    private String bio;
    private String avatarUrl;
}
```

6.4 @ManyToMany

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany
    @JoinTable(
        name = "student_courses",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
```

```java
    private Set<Course> courses = new HashSet<>();
}

@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students = new HashSet<>();
}
```

6.5 Cascade Types

Cascade Type	Effect	When to Use
`PERSIST`	Save child when parent is saved	Parent-child relationships
`MERGE`	Update child when parent is updated	Almost always with PERSIST
`REMOVE`	Delete child when parent is deleted	Owned relationships
`REFRESH`	Refresh child when parent is refreshed	Rare
`DETACH`	Detach child when parent is detached	Rare
`ALL`	All of the above	Strong parent-child ownership

6.6 Owning Side vs Inverse Side

Concept	Owning Side	Inverse Side
Has the FK	Yes (JoinColumn)	No
Controls relationship	Yes	No -- uses `mappedBy`
Annotation	`@JoinColumn`	`mappedBy = "fieldName"`
Example	`Order.customer` (has customer_id FK)	`Customer.orders` (no FK)

Rule: The side with the foreign key is always the owning side. In `@ManyToOne`, the "Many" side always owns.

Part 7: Fetching Strategies

7.1 Lazy vs Eager Loading

Lazy Loading (Default for Collections)

```
@OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
private List<Order> orders;
// SQL: Only loads Customer. Orders loaded ONLY when accessed.
// SELECT * FROM customers WHERE id = ?
// ... later when customer.getOrders() is called:
// SELECT * FROM orders WHERE customer_id = ?
```

Eager Loading

```
@ManyToOne(fetch = FetchType.EAGER)
private Customer customer;
// SQL: Loads Customer AND related data immediately via JOIN
// SELECT c.*, o.* FROM customers c JOIN orders o ON ...
```

Defaults

Association	Default Fetch Type
`@ManyToOne`	EAGER
`@OneToOne`	EAGER
`@OneToMany`	LAZY
`@ManyToMany`	LAZY

Best Practice: Always set `@ManyToOne(fetch = FetchType.LAZY)` and `@OneToOne(fetch = FetchType.LAZY)`. Override defaults.

7.2 The N+1 Query Problem

The most common Hibernate performance problem in production.

```
// N+1 Problem Example:
List<Customer> customers = customerRepository.findAll();
// Query 1: SELECT * FROM customers  (returns 100 customers)

for (Customer c : customers) {
    System.out.println(c.getOrders().size());
    // Query 2: SELECT * FROM orders WHERE customer_id = 1
    // Query 3: SELECT * FROM orders WHERE customer_id = 2
    // ...
    // Query 101: SELECT * FROM orders WHERE customer_id = 100
}
// TOTAL: 1 + 100 = 101 queries! Extremely slow!
```

Solution 1: JOIN FETCH (JPQL)

```
@Query("SELECT c FROM Customer c JOIN FETCH c.orders")
List<Customer> findAllWithOrders();
// Single query: SELECT c.*, o.* FROM customers c JOIN orders o ON ...
```

Solution 2: @EntityGraph

```
@EntityGraph(attributePaths = {"orders"})
List<Customer> findAll();
// Generates LEFT JOIN FETCH automatically
```

Solution 3: @BatchSize

```
@OneToMany(mappedBy = "customer")
@BatchSize(size = 25)
private List<Order> orders;
// Instead of 100 queries, fires 4 queries (100/25 = 4 batches):
// SELECT * FROM orders WHERE customer_id IN (1,2,3,...,25)
// SELECT * FROM orders WHERE customer_id IN (26,27,...,50)
// ...
```

Part 8: Hibernate Query Methods

8.1 HQL (Hibernate Query Language)

```
// HQL operates on ENTITIES, not tables
// Hibernate-specific (not portable to other JPA providers)

// Select
List<User> users = session.createQuery(
    "FROM User u WHERE u.status = :status", User.class)
    .setParameter("status", UserStatus.ACTIVE)
    .getResultList();

// Update
int updated = session.createQuery(
    "UPDATE User u SET u.status = :newStatus WHERE u.lastLogin < :cutoff")
    .setParameter("newStatus", UserStatus.INACTIVE)
    .setParameter("cutoff", LocalDateTime.now().minusDays(90))
    .executeUpdate();
```

8.2 JPQL (Java Persistence Query Language)

```
// JPQL is the JPA standard -- portable across ORM providers
// Very similar to HQL

@Query("SELECT u FROM User u WHERE u.email LIKE %:domain")
List<User> findByEmailDomain(@Param("domain") String domain);

@Query("SELECT NEW com.app.dto.UserSummaryDTO(u.id, u.name, u.email) " +
       "FROM User u WHERE u.status = :status")
List<UserSummaryDTO> findUserSummaries(@Param("status") UserStatus status);

// Aggregations
@Query("SELECT u.department, COUNT(u), AVG(u.salary) " +
       "FROM User u GROUP BY u.department")
List<Object[]> getDepartmentStats();
```

8.3 Native SQL

```
// Use actual SQL -- not portable, but supports DB-specific features
@Query(value = "SELECT * FROM users WHERE email LIKE %?1 " +
               "ORDER BY created_at DESC LIMIT 10",
       nativeQuery = true)
List<User> findRecentByEmailDomain(String domain);

// With result mapping
@Query(value = "SELECT u.id, u.name, COUNT(o.id) as order_count " +
               "FROM users u LEFT JOIN orders o ON u.id = o.user_id " +
               "GROUP BY u.id, u.name",
       nativeQuery = true)
List<Object[]> getUserOrderCounts();
```

8.4 Comparison

Feature	HQL	JPQL	Native SQL
Standard	Hibernate-specific	JPA standard	Database-specific
Operates on	Entities	Entities	Tables
Portability	Hibernate only	Any JPA provider	Not portable
DB functions	Some	Limited	Full support
Complex queries	Good	Good	Best
Use when	Hibernate features needed	Default choice	Complex/optimized SQL

Part 9: Transaction Management

What is Transaction Management in Spring?

Spring's `@Transactional` annotation provides **declarative transaction management** -- you annotate a method, and Spring automatically wraps it in a database transaction. No manual `begin/commit/rollback` code needed.

Why @Transactional Over Manual Transactions

Manual Transaction	@Transactional
10+ lines of try/catch/finally	Single annotation
Easy to forget rollback	Automatic rollback on exceptions
Scattered across codebase	Clean separation of concerns
Hard to test	Mockable, testable

Step-by-Step: How @Transactional Works Internally

```
1. Spring creates a proxy around the @Service class
2. When a @Transactional method is called:
   a. Proxy intercepts the call
   b. Opens a new transaction (or joins existing)
   c. Calls the actual method
   d. If method succeeds -> COMMIT
   e. If RuntimeException is thrown -> ROLLBACK
   f. If checked exception is thrown -> COMMIT (by default!)
```

> ***Critical Gotcha:*** *@Transactional does NOT work on* `private` *methods or when calling a @Transactional method from within the* ***same class*** *(self-invocation bypasses the proxy).*

9.1 ACID Properties

Property	Meaning	Example

Atomicity	All or nothing -- entire transaction succeeds or rolls back	Transfer $100: debit AND credit must both succeed
Consistency	Database moves from one valid state to another	Balance never goes negative if constraint exists
Isolation	Concurrent transactions don't interfere	Two users buying last item -- only one succeeds
Durability	Committed data survives crashes	After commit, data persists even if server restarts

9.2 Spring @Transactional

When to Use @Transactional

- **Service layer methods** that modify data (INSERT, UPDATE, DELETE)
- **Multi-step operations** where all steps must succeed or fail together
- Methods that read data and need **snapshot consistency** (`readOnly = true`)

When NOT to Use @Transactional

- **Controller methods** -- transactions should be scoped at the service layer
- **Very long-running operations** -- holding a transaction for minutes blocks database resources
- **External API calls** within a transaction -- the API call cannot be rolled back

```
@Service
public class OrderService {

    @Transactional // Wraps method in a database transaction
    public Order placeOrder(OrderRequest request) {
        // 1. Validate inventory
        Product product = productRepository.findById(request.getProductId())
            .orElseThrow(() -> new
ProductNotFoundException(request.getProductId()));

        if (product.getStock() < request.getQuantity()) {
            throw new InsufficientStockException();
        }

        // 2. Reduce stock
        product.setStock(product.getStock() - request.getQuantity());

        // 3. Create order
        Order order = new Order();
```

```
        order.setProduct(product);
        order.setQuantity(request.getQuantity());
        order.setTotalPrice(product.getPrice()
            .multiply(BigDecimal.valueOf(request.getQuantity())));

        // 4. Save and return
        return orderRepository.save(order);
        // If ANY step throws an exception, ALL changes are rolled back
    }
}
```

9.3 Transaction Propagation

What is Transaction Propagation?

Propagation defines what happens when a @Transactional method calls another @Transactional method. Should they share the same transaction? Create a new one? Throw an error?

Why This Matters in Production

Consider: An order placement method calls an audit logging method. If the order fails and rolls back, should the audit log also roll back? Usually NO -- you want the audit log to persist. This is where `REQUIRES_NEW` propagation saves you.

Propagation	Behavior	Use Case
`REQUIRED` (default)	Join existing or create new	Most service methods
`REQUIRES_NEW`	Always create new (suspend existing)	Audit logging that must persist even if parent fails
`SUPPORTS`	Join existing or run non-transactional	Read-only queries
`NOT_SUPPORTED`	Suspend existing, run non-transactional	Non-critical operations
`MANDATORY`	Must have existing, throw if none	Methods that should never be called directly
`NEVER`	Throw if transaction exists	Methods that must not run in a transaction
`NESTED`	Create savepoint within existing	Partial rollbacks

```java
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logAuditEvent(String action, String details) {
    // This runs in its OWN transaction
    // Even if the calling method rolls back, audit log is preserved
    AuditLog log = new AuditLog(action, details, Instant.now());
    auditRepository.save(log);
}
```

9.4 Isolation Levels

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
READ_UNCOMMITTED	Yes	Yes	Yes	Fastest
READ_COMMITTED	No	Yes	Yes	Good (PostgreSQL default)
REPEATABLE_READ	No	No	Yes	Medium (MySQL default)
SERIALIZABLE	No	No	No	Slowest

```java
@Transactional(isolation = Isolation.READ_COMMITTED)
public BigDecimal getAccountBalance(Long accountId) { ... }
```

9.5 Read-Only Transactions

```java
@Transactional(readOnly = true) // Performance optimization
public List<User> getAllActiveUsers() {
    return userRepository.findByStatus(UserStatus.ACTIVE);
    // Hibernate skips dirty checking -- faster!
    // Spring may route to read replica if configured
}
```

Part 10: Spring Data JPA Introduction

10.1 What is Spring Data JPA?

Spring Data JPA is an abstraction layer on top of JPA/Hibernate that eliminates boilerplate repository code. You define an interface, and Spring generates the implementation at runtime.

Why Spring Data JPA Exists

Without Spring Data JPA, every entity requires a repository class with 5+ methods of repetitive code (find, save, delete, update, count). Spring Data JPA generates these implementations **automatically** from just an interface declaration.

Design Patterns Used

Pattern	Where Used
Repository Pattern	Each interface represents a data access object for one entity
Proxy Pattern	Spring creates a runtime proxy implementing the interface
Template Method	`SimpleJpaRepository` provides default implementations
Strategy Pattern	Query derivation strategies (method name, @Query, native)

```java
// WITHOUT Spring Data JPA -- manual repository
@Repository
public class UserRepositoryImpl {

    @PersistenceContext
    private EntityManager em;

    public User findById(Long id) {
        return em.find(User.class, id);
    }

    public List<User> findByStatus(UserStatus status) {
        return em.createQuery(
            "SELECT u FROM User u WHERE u.status = :status", User.class)
            .setParameter("status", status)
            .getResultList();
    }
```

```java
    public User save(User user) {
        if (user.getId() == null) {
            em.persist(user);
            return user;
        }
        return em.merge(user);
    }

    public void delete(User user) {
        em.remove(em.contains(user) ? user : em.merge(user));
    }

    public List<User> findAll() {
        return em.createQuery("SELECT u FROM User u", User.class)
            .getResultList();
    }
}

// WITH Spring Data JPA -- just an interface!
public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByStatus(UserStatus status);
    // That's it! Spring generates everything else automatically.
}
```

10.2 How Spring Data JPA Works

1. You define an interface extending `JpaRepository`
2. Spring Boot scans for repository interfaces at startup
3. Spring Data creates a **proxy** implementation at runtime
4. The proxy delegates to `SimpleJpaRepository` (default implementation)
5. `SimpleJpaRepository` uses `EntityManager` internally

Part 11: Spring Data JPA Architecture

11.1 Architecture Diagram

```
+-------------------------------------------------------------+
|                      Spring Boot Application                 |
|                                                              |
|  +------------------+                                        |
|  |   Controller     |  @RestController                       |
|  |   (REST API)     |  Handles HTTP requests                 |
|  +--------+---------+                                        |
|           |                                                  |
|           v                                                  |
|  +------------------+                                        |
|  |    Service        |  @Service @Transactional               |
|  |   (Business       |  Business logic and                    |
|  |    Logic)         |  transaction boundaries                |
|  +--------+---------+                                        |
|           |                                                  |
|           v                                                  |
|  +------------------+                                        |
|  |   Repository     |  JpaRepository interface               |
|  |   (Data Access)  |  Auto-generated implementation         |
|  +--------+---------+                                        |
|           |                                                  |
|           v                                                  |
|  +------------------+                                        |
|  | SimpleJpaRepo    |  Spring Data's default implementation  |
|  | (Generated Proxy)|  Uses EntityManager internally         |
|  +--------+---------+                                        |
|           |                                                  |
|           v                                                  |
|  +------------------+  +-------------------+                 |
|  | EntityManager    |  | Hibernate Session |                 |
|  | (JPA Standard)   |  | (Implementation)  |                 |
|  +--------+---------+  +-------------------+                 |
|           |                                                  |
|           v                                                  |
|  +------------------+                                        |
|  |   JDBC / HikariCP|  Connection pooling                    |
|  +--------+---------+                                        |
|           |                                                  |
|           v                                                  |
```

```
|  +------------------+                                          |
|  |   Database       |  PostgreSQL, MySQL, Oracle               |
|  +------------------+                                          |
+----------------------------------------------------------------+
```

11.2 The Stack

Layer	Technology	Role
API	Spring MVC / WebFlux	REST endpoints
Service	Spring @Service	Business logic, transactions
Repository	Spring Data JPA	Data access abstraction
ORM	Hibernate 6	Object-relational mapping
Connection Pool	HikariCP	JDBC connection management
Database	PostgreSQL/MySQL	Data storage

Part 12: Spring Data JPA Repositories

12.1 Repository Hierarchy

```
Repository (Marker interface)
    |
CrudRepository (CRUD operations)
    |
ListCrudRepository (List-returning CRUD)
    |
PagingAndSortingRepository (+ Pagination, Sorting)
    |
JpaRepository (+ JPA-specific: flush, batch, example queries)
```

12.2 CrudRepository

```java
// Provides basic CRUD
public interface CrudRepository<T, ID> extends Repository<T, ID> {
    <S extends T> S save(S entity);
    Optional<T> findById(ID id);
    boolean existsById(ID id);
    Iterable<T> findAll();
    long count();
    void deleteById(ID id);
    void delete(T entity);
}
```

12.3 JpaRepository (Recommended)

```java
// Extends CrudRepository + adds JPA features
public interface JpaRepository<T, ID> extends
        ListCrudRepository<T, ID>,
        ListPagingAndSortingRepository<T, ID>,
        QueryByExampleExecutor<T> {
```

```
    void flush();
    <S extends T> S saveAndFlush(S entity);
    void deleteAllInBatch();
    List<T> findAll(Sort sort);
    Page<T> findAll(Pageable pageable);
}

// Usage:
public interface ProductRepository extends JpaRepository<Product, Long> {
    // Inherits: save, findById, findAll, count, delete,
    //           flush, saveAndFlush, pagination, sorting, etc.
}
```

12.4 When to Use Which

Repository	Use When
`CrudRepository`	Simple CRUD, no pagination needed
`PagingAndSortingRepository`	Need pagination but no JPA features
`JpaRepository`	Default choice for Spring Boot apps

Part 13: Query Methods in Spring Data JPA

Three Ways to Define Queries in Spring Data JPA

Spring Data JPA provides three strategies for creating queries, from simplest to most powerful:

Strategy	When to Use	Example
Derived query (method name)	Simple conditions (1-3 fields)	`findByName(String name)`
@Query (JPQL)	Complex queries, joins, aggregations	`@Query("SELECT u FROM User u WHERE...")`
Native query	Database-specific features, complex analytics	`@Query(value = "SELECT...", nativeQuery = true)`

> ***Rule of Thumb:*** *Start with derived queries. Move to @Query when method names become unreadable (more than 3 conditions). Use native queries only when you need database-specific syntax.*

13.1 Method Name Query Derivation

How It Works

Spring Data parses method names and generates JPQL automatically. The method name IS the query -- Spring breaks it into parts:

```
public interface UserRepository extends JpaRepository<User, Long> {

    // SELECT u FROM User u WHERE u.name = ?1
    List<User> findByName(String name);

    // SELECT u FROM User u WHERE u.email = ?1
    Optional<User> findByEmail(String email);

    // SELECT u FROM User u WHERE u.status = ?1 AND u.role = ?2
    List<User> findByStatusAndRole(UserStatus status, Role role);

    // SELECT u FROM User u WHERE u.name LIKE %?1%
    List<User> findByNameContaining(String keyword);

    // SELECT u FROM User u WHERE u.age BETWEEN ?1 AND ?2
```

```
    List<User> findByAgeBetween(int min, int max);

    // SELECT u FROM User u WHERE u.status IN ?1
    List<User> findByStatusIn(Collection<UserStatus> statuses);

    // SELECT u FROM User u WHERE u.deletedAt IS NULL
    List<User> findByDeletedAtIsNull();

    // SELECT u FROM User u ORDER BY u.createdAt DESC
    List<User> findAllByOrderByCreatedAtDesc();

    // COUNT
    long countByStatus(UserStatus status);

    // EXISTS
    boolean existsByEmail(String email);

    // DELETE
    void deleteByStatus(UserStatus status);

    // LIMIT
    List<User> findTop10ByOrderByCreatedAtDesc();
    Optional<User> findFirstByOrderByCreatedAtDesc();
}
```

13.2 Query Keywords Reference

Keyword	SQL Equivalent	Example
`And`	AND	`findByNameAndEmail`
`Or`	OR	`findByNameOrEmail`
`Between`	BETWEEN	`findByAgeBetween`
`LessThan`	<	`findByAgeLessThan`
`GreaterThan`	>	`findByAgeGreaterThan`
`Like`	LIKE	`findByNameLike`
`Containing`	LIKE %...%	`findByNameContaining`
`StartingWith`	LIKE ...%	`findByNameStartingWith`
`In`	IN (...)	`findByStatusIn`

`IsNull`	IS NULL	`findByDeletedAtIsNull`
`IsNotNull`	IS NOT NULL	`findByEmailIsNotNull`
`OrderBy`	ORDER BY	`findByStatusOrderByNameAsc`
`Not`	`<>`	`findByStatusNot`
`True` / `False`	= true/false	`findByActiveTrue`

Part 14: Custom Queries

14.1 @Query with JPQL

```
public interface OrderRepository extends JpaRepository<Order, Long> {

    // JPQL -- uses entity names, not table names
    @Query("SELECT o FROM Order o WHERE o.customer.id = :customerId " +
           "AND o.status = :status ORDER BY o.createdAt DESC")
    List<Order> findCustomerOrdersByStatus(
        @Param("customerId") Long customerId,
        @Param("status") OrderStatus status);

    // Projection -- return DTO instead of entity
    @Query("SELECT NEW com.app.dto.OrderSummary(o.id, o.totalAmount, " +
           "o.status, o.createdAt) FROM Order o " +
           "WHERE o.customer.id = :customerId")
    List<OrderSummary> findOrderSummaries(@Param("customerId") Long
customerId);

    // Aggregation
    @Query("SELECT o.status, COUNT(o), SUM(o.totalAmount) " +
           "FROM Order o GROUP BY o.status")
    List<Object[]> getOrderStatistics();

    // Update query
    @Modifying
    @Transactional
    @Query("UPDATE Order o SET o.status = :newStatus " +
           "WHERE o.status = :oldStatus AND o.createdAt < :cutoff")
    int bulkUpdateStatus(@Param("oldStatus") OrderStatus oldStatus,
                         @Param("newStatus") OrderStatus newStatus,
                         @Param("cutoff") LocalDateTime cutoff);
}
```

14.2 Native Queries

```
// When you need database-specific features
@Query(value = "SELECT * FROM orders WHERE total_amount > :min " +
```

```
                "ORDER BY created_at DESC LIMIT :limit",
        nativeQuery = true)
List<Order> findExpensiveRecentOrders(@Param("min") BigDecimal min,
                                      @Param("limit") int limit);

// Complex analytics
@Query(value = """
    SELECT DATE(o.created_at) as order_date,
           COUNT(*) as total_orders,
           SUM(o.total_amount) as revenue
    FROM orders o
    WHERE o.created_at >= :startDate
    GROUP BY DATE(o.created_at)
    ORDER BY order_date
    """, nativeQuery = true)
List<Object[]> getDailyRevenue(@Param("startDate") LocalDate startDate);
```

Part 15: Pagination and Sorting

15.1 Pageable

```
// Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    Page<Product> findByCategory(String category, Pageable pageable);
}

// Service
@Transactional(readOnly = true)
public Page<ProductDTO> getProducts(String category, int page, int size) {
    Pageable pageable = PageRequest.of(page, size,
Sort.by("name").ascending());
    Page<Product> products = productRepository.findByCategory(category,
pageable);
    return products.map(this::toDTO); // Map entity to DTO
}

// Controller
@GetMapping("/api/products")
public Page<ProductDTO> listProducts(
        @RequestParam(defaultValue = "") String category,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size,
        @RequestParam(defaultValue = "name,asc") String[] sort) {

    Sort sortObj = Sort.by(Sort.Direction.fromString(sort[1]), sort[0]);
    return productService.getProducts(category, page, size);
}
```

15.2 Page Response

```
{
  "content": [ ... ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": { "sorted": true, "orders": [{"property": "name", "direction":
```

```
"ASC"}] }
  },
  "totalElements": 156,
  "totalPages": 8,
  "last": false,
  "first": true,
  "numberOfElements": 20
}
```

15.3 Sorting

```
// Multiple sort criteria
Sort sort = Sort.by(
    Sort.Order.desc("priority"),
    Sort.Order.asc("name")
);
List<Product> sorted = productRepository.findAll(sort);
```

Part 16: Spring Data JPA Specifications

What are Specifications?

Specifications allow you to build **dynamic, composable queries** at runtime. Instead of writing separate repository methods for every possible filter combination, you create reusable building blocks that can be combined with AND/OR.

Why Specifications Over Multiple Repository Methods

Imagine a product search page with 6 optional filters (category, price range, keyword, brand, rating, availability). Without Specifications, you'd need $2^6 = 64$ repository methods for every possible combination. With Specifications, you need just 6 reusable building blocks.

When to Use Specifications

- **Search/filter pages** with multiple optional criteria
- **Admin dashboards** where users combine filters dynamically
- When query conditions are known only at runtime

When NOT to Use Specifications

- Simple, fixed queries -- use derived query methods instead
- Complex analytics -- use native queries or views
- When there are only 1-2 filter conditions

Design Pattern: Specification Pattern (Domain-Driven Design)

This implements the **Specification pattern** from DDD -- each specification encapsulates a single business rule that can be combined with `and()`, `or()`, `not()`.

16.1 Dynamic Queries with Specifications

```
// Repository must extend JpaSpecificationExecutor
public interface ProductRepository extends JpaRepository<Product, Long>,
        JpaSpecificationExecutor<Product> {
}

// Specification class
public class ProductSpecifications {

    public static Specification<Product> hasCategory(String category) {
```

```java
        return (root, query, cb) ->
            category == null ? null :
            cb.equal(root.get("category"), category);
    }

    public static Specification<Product> priceBetween(
            BigDecimal min, BigDecimal max) {
        return (root, query, cb) -> {
            if (min == null && max == null) return null;
            if (min != null && max != null)
                return cb.between(root.get("price"), min, max);
            if (min != null)
                return cb.greaterThanOrEqualTo(root.get("price"), min);
            return cb.lessThanOrEqualTo(root.get("price"), max);
        };
    }

    public static Specification<Product> nameContains(String keyword) {
        return (root, query, cb) ->
            keyword == null ? null :
            cb.like(cb.lower(root.get("name")),
                    "%" + keyword.toLowerCase() + "%");
    }

    public static Specification<Product> isActive() {
        return (root, query, cb) ->
            cb.equal(root.get("active"), true);
    }
}

// Service -- compose specifications dynamically
@Transactional(readOnly = true)
public Page<Product> searchProducts(ProductSearchRequest req, Pageable
pageable) {
    Specification<Product> spec = Specification
        .where(ProductSpecifications.isActive())
        .and(ProductSpecifications.hasCategory(req.getCategory()))
        .and(ProductSpecifications.priceBetween(req.getMinPrice(),
req.getMaxPrice()))
        .and(ProductSpecifications.nameContains(req.getKeyword()));

    return productRepository.findAll(spec, pageable);
}
```

Part 17: Performance Optimization

17.1 Batch Fetching

```yaml
// application.yml
spring:
  jpa:
    properties:
      hibernate:
        default_batch_fetch_size: 25
        jdbc:
          batch_size: 50
        order_inserts: true
        order_updates: true
```

```java
// Entity-level batch size
@Entity
public class Customer {
    @OneToMany(mappedBy = "customer")
    @BatchSize(size = 25) // Fetch orders in batches of 25
    private List<Order> orders;
}
// Instead of N queries, generates: SELECT * FROM orders WHERE customer_id IN
(?,?,?,...,?)
```

17.2 Connection Pooling (HikariCP)

```yaml
spring:
  datasource:
    hikari:
      maximum-pool-size: 20        # Max connections
      minimum-idle: 5              # Min idle connections
      idle-timeout: 300000         # 5 min idle timeout
      connection-timeout: 20000    # 20 sec connection timeout
```

```yaml
      max-lifetime: 1800000        # 30 min max connection lifetime
      leak-detection-threshold: 60000  # Detect connection leaks
```

17.3 DTO Projections (Avoid Fetching Full Entities)

```java
// Interface-based projection
public interface UserSummary {
    Long getId();
    String getName();
    String getEmail();
}

public interface UserRepository extends JpaRepository<User, Long> {
    List<UserSummary> findByStatus(UserStatus status);
    // Generates: SELECT u.id, u.name, u.email FROM users u WHERE u.status = ?
    // NOT: SELECT * FROM users ... (avoids loading unnecessary columns)
}

// Class-based projection (DTO)
public record UserDTO(Long id, String name, String email) {}

@Query("SELECT new com.app.dto.UserDTO(u.id, u.name, u.email) " +
       "FROM User u WHERE u.status = :status")
List<UserDTO> findUserDTOs(@Param("status") UserStatus status);
```

17.4 Bulk Operations

```java
// BAD: Loading and saving one by one
for (Long id : userIds) {
    User user = userRepository.findById(id).get();
    user.setStatus(UserStatus.INACTIVE);
    userRepository.save(user); // N SELECT + N UPDATE queries!
}

// GOOD: Bulk update with @Query
@Modifying
@Transactional
@Query("UPDATE User u SET u.status = :status WHERE u.id IN :ids")
int bulkUpdateStatus(@Param("ids") List<Long> ids,
```

```
                        @Param("status") UserStatus status);
// Single UPDATE query regardless of list size
```

Part 18: Hibernate Caching

18.1 First-Level Cache (Session Cache)

What: Built-in, always-on cache within each Hibernate Session (EntityManager).

Scope: Per session (per transaction in Spring).

How it works:

```
@Transactional
public void demonstrateFirstLevelCache() {
    // Query 1: Hits database
    User u1 = userRepository.findById(1L).get();
    // SQL: SELECT * FROM users WHERE id = 1

    // Query 2: Returns from cache -- NO database hit!
    User u2 = userRepository.findById(1L).get();
    // No SQL executed!

    assert u1 == u2; // Same object reference (identity guarantee)
}
```

18.2 Second-Level Cache

What: Shared cache across all Sessions. Optional, must be configured.

Scope: SessionFactory-level (application-wide).

```
+---------------------------------+
|     Application (Multiple       |
|     Concurrent Requests)        |
+---------------------------------+
   |          |           |
   v          v           v
+------+  +------+    +------+
|Sess 1|  |Sess 2|    |Sess 3|  <-- Each has own 1st level cache
+------+  +------+    +------+
     \       |        /
      +------+-------+
```

```
      | 2nd Level     |  <-- Shared across ALL sessions
      | Cache         |
      | (EhCache/     |
      |  Redis/       |
      |  Hazelcast)   |
      +-------+-------+
              |
              v
      +-------+-------+
      |   Database     |
      +---------------+
```

Configuration

```
<!-- pom.xml -->
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jcache</artifactId>
</dependency>
<dependency>
    <groupId>org.ehcache</groupId>
    <artifactId>ehcache</artifactId>
</dependency>
```

```
# application.yml
spring:
  jpa:
    properties:
      hibernate:
        cache:
          use_second_level_cache: true
          region.factory_class: jcache
        javax:
          cache:
            provider: org.ehcache.jsr107.EhcacheCachingProvider
```

```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
    private String name;
    private BigDecimal price;
}
// Now Product entities are cached in 2nd level cache
// Subsequent findById() calls return from cache, not DB
```

Cache Strategies

Strategy	Description	Use When
`READ_ONLY`	Never changes after creation	Reference data, countries, currencies
`READ_WRITE`	Reads and updates	Most entities
`NONSTRICT_READ_WRITE`	Eventual consistency	Data where slight staleness is OK
`TRANSACTIONAL`	Full transaction support	JTA environments

18.3 Query Cache

```yaml
spring:
  jpa:
    properties:
      hibernate:
        cache:
          use_query_cache: true
```

```
@QueryHints(@QueryHint(name = "org.hibernate.cacheable", value = "true"))
List<Product> findByCategory(String category);
// Results of this query are cached
// Invalidated when any Product entity changes
```

Part 19: Common Performance Problems

19.1 N+1 Query Problem

Problem: Loading a list of entities, then lazy-loading a relationship for each one.

```
// This generates 1 + N queries:
List<Customer> customers = customerRepository.findAll(); // 1 query
for (Customer c : customers) {
    c.getOrders().size(); // N queries (one per customer)
}
```

Solutions:

Solution	Method	Query Count
JOIN FETCH	`@Query("SELECT c FROM Customer c JOIN FETCH c.orders")`	1
@EntityGraph	`@EntityGraph(attributePaths = {"orders"})`	1
@BatchSize	`@BatchSize(size = 25)` on collection	N/25
DTO Projection	Select only needed columns	1

19.2 Lazy Loading Exceptions

Problem: `LazyInitializationException` when accessing lazy-loaded data outside Session.

```
// BAD: Session is closed when controller tries to access orders
@GetMapping("/customers/{id}")
public Customer getCustomer(@PathVariable Long id) {
    Customer c = customerService.findById(id); // Session closes after service
    c.getOrders().size(); // LazyInitializationException!
}
```

Solutions:

```java
// Solution 1: Fetch eagerly in the query
@Query("SELECT c FROM Customer c JOIN FETCH c.orders WHERE c.id = :id")
Customer findByIdWithOrders(@Param("id") Long id);

// Solution 2: @Transactional on the service method
@Transactional(readOnly = true)
public CustomerDTO getCustomerWithOrders(Long id) {
    Customer c = customerRepository.findById(id).orElseThrow();
    return new CustomerDTO(c, c.getOrders()); // Access while session is open
}

// Solution 3: EntityGraph
@EntityGraph(attributePaths = {"orders"})
Optional<Customer> findById(Long id);
```

19.3 Slow Queries

Diagnosis:

```yaml
spring:
  jpa:
    properties:
      hibernate:
        generate_statistics: true  # Log query statistics
        format_sql: true
    show-sql: true

logging:
  level:
    org.hibernate.SQL: DEBUG
    org.hibernate.orm.jdbc.bind: TRACE  # Show parameter values
```

Solutions:

- Add database indexes for frequently queried columns
- Use DTO projections instead of full entities
- Add `@BatchSize` for collections
- Use pagination for large result sets
- Avoid `SELECT *` via projections

19.4 Memory Issues

```
// BAD: Loading 1 million entities into memory
List<User> all = userRepository.findAll(); // OutOfMemoryError!

// GOOD: Process in pages
int page = 0;
Page<User> batch;
do {
    batch = userRepository.findAll(PageRequest.of(page, 500));
    batch.getContent().forEach(this::processUser);
    entityManager.clear(); // Clear persistence context to free memory
    page++;
} while (batch.hasNext());
```

Part 20: Real Production Scenario

20.1 User Management System

Architecture

```
+-----------+     +-------------+     +--------------+     +----------+
|  Client   | --> | Controller  | --> |   Service    | --> |Repository|
| (React)   |     | (REST API)  |     | (@Transact.) |     | (JPA)    |
+-----------+     +-------------+     +--------------+     +----------+
                                                                 |
                                                                 v
                                                            +----------+
                                                            | Database |
                                                            |PostgreSQL|
                                                            +----------+
```

Entity

```java
@Entity
@Table(name = "users",
    indexes = @Index(name = "idx_user_email", columnList = "email"))
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE,
        generator = "user_seq")
    @SequenceGenerator(name = "user_seq",
        sequenceName = "user_sequence", allocationSize = 50)
    private Long id;

    @Column(nullable = false, length = 100)
    private String firstName;

    @Column(nullable = false, length = 100)
    private String lastName;

    @Column(nullable = false, unique = true)
    private String email;
```

```java
    @Column(nullable = false)
    private String passwordHash;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private UserStatus status = UserStatus.ACTIVE;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role = Role.USER;

    @Embedded
    private Address address;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<UserSession> sessions = new ArrayList<>();

    @Column(updatable = false)
    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = createdAt;
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

Repository

```java
public interface UserRepository extends JpaRepository<User, Long>,
        JpaSpecificationExecutor<User> {

    Optional<User> findByEmail(String email);

    boolean existsByEmail(String email);

    @Query("SELECT u FROM User u WHERE u.status = :status")
```

```
    Page<User> findByStatus(@Param("status") UserStatus status,
                            Pageable pageable);

    @EntityGraph(attributePaths = {"sessions"})
    Optional<User> findWithSessionsById(Long id);

    @Modifying
    @Query("UPDATE User u SET u.status = 'INACTIVE' " +
           "WHERE u.updatedAt < :cutoff AND u.status = 'ACTIVE'")
    int deactivateInactiveUsers(@Param("cutoff") LocalDateTime cutoff);

    @Query("SELECT new com.app.dto.UserSummaryDTO(u.id, u.firstName, " +
           "u.lastName, u.email, u.status) FROM User u")
    Page<UserSummaryDTO> findAllSummaries(Pageable pageable);
}
```

Service

```
@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Transactional
    public UserDTO createUser(CreateUserRequest request) {
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new DuplicateEmailException(request.getEmail());
        }

        User user = new User();
        user.setFirstName(request.getFirstName());
        user.setLastName(request.getLastName());
        user.setEmail(request.getEmail());
        user.setPasswordHash(passwordEncoder.encode(request.getPassword()));
        user.setRole(request.getRole());

        User saved = userRepository.save(user);
        return UserDTO.from(saved);
    }

    @Transactional(readOnly = true)
    public Page<UserSummaryDTO> listUsers(int page, int size) {
        return userRepository.findAllSummaries(
```

```
            PageRequest.of(page, size, Sort.by("createdAt").descending()));
    }

    @Transactional
    public UserDTO updateUser(Long id, UpdateUserRequest request) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
        user.setFirstName(request.getFirstName());
        user.setLastName(request.getLastName());
        // Dirty checking auto-saves changes on commit
        return UserDTO.from(user);
    }
}
```

Controller

```
@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @PostMapping
    public ResponseEntity<UserDTO> createUser(@Valid @RequestBody
CreateUserRequest req) {
        UserDTO user = userService.createUser(req);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }

    @GetMapping
    public Page<UserSummaryDTO> listUsers(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size) {
        return userService.listUsers(page, size);
    }

    @PutMapping("/{id}")
    public UserDTO updateUser(@PathVariable Long id,
                              @Valid @RequestBody UpdateUserRequest req) {
        return userService.updateUser(id, req);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
```

```
        userService.deleteUser(id);
        return ResponseEntity.noContent().build();
    }
}
```

Part 21: Microservices Data Strategy

21.1 Database Per Service Pattern

```
+------------------+    +------------------+    +------------------+
|  User Service    |    |  Order Service   |    | Payment Service  |
|  +-----------+   |    |  +-----------+   |    |  +-----------+   |
|  | Spring    |   |    |  | Spring    |   |    |  | Spring    |   |
|  | Data JPA  |   |    |  | Data JPA  |   |    |  | Data JPA  |   |
|  +-----------+   |    |  +-----------+   |    |  +-----------+   |
|       |          |    |       |          |    |       |          |
|  +-----------+   |    |  +-----------+   |    |  +-----------+   |
|  | User DB   |   |    |  | Order DB  |   |    |  | Payment DB|   |
|  | PostgreSQL|   |    |  | MySQL     |   |    |  | PostgreSQL|   |
|  +-----------+   |    |  +-----------+   |    |  +-----------+   |
+------------------+    +------------------+    +------------------+
```

Key Rules:

1. Each service owns its database -- no direct cross-service DB access
2. Services communicate via APIs or events, never via shared DB
3. Each service has its own `application.yml` with its own datasource

21.2 Saga Pattern (Distributed Transactions)

```
Order Saga (Place Order):

1. Order Service: Create order (PENDING)
         |
         v
2. Payment Service: Reserve payment
         |
    Success? ----No----> Compensate: Cancel order
         |
        Yes
         |
         v
3. Inventory Service: Reserve stock
         |
```

```
    Success? ----No----> Compensate: Refund payment, Cancel order
         |
        Yes
         |
         v
4. Order Service: Confirm order (CONFIRMED)
```

```java
// Orchestration-based Saga
@Service
public class OrderSagaOrchestrator {

    @Transactional
    public OrderResult placeOrder(OrderRequest request) {
        // Step 1: Create order
        Order order = orderService.create(request);

        try {
            // Step 2: Reserve payment
            PaymentResult payment = paymentClient.reserve(
                order.getId(), order.getTotalAmount());

            // Step 3: Reserve inventory
            InventoryResult inventory = inventoryClient.reserve(
                order.getItems());

            // Step 4: Confirm
            order.confirm();
            return OrderResult.success(order);

        } catch (PaymentException e) {
            order.cancel("Payment failed");
            throw e;
        } catch (InventoryException e) {
            paymentClient.refund(order.getId()); // Compensate
            order.cancel("Inventory unavailable");
            throw e;
        }
    }
}
```

21.3 Eventual Consistency with Events

```java
// Order Service publishes event after saving
@Transactional
public Order createOrder(OrderRequest request) {
    Order order = orderRepository.save(new Order(request));

    // Publish event via Kafka
    kafkaTemplate.send("order-events",
        new OrderCreatedEvent(order.getId(), order.getCustomerId(),
            order.getTotalAmount()));

    return order;
}

// Inventory Service consumes event
@KafkaListener(topics = "order-events")
public void handleOrderCreated(OrderCreatedEvent event) {
    inventoryService.reserveStock(event.getOrderId(), event.getItems());
}
```

Part 22: Build It Yourself -- User Management with JPA

> ***Goal:*** *Build a Spring Boot User Management REST API using Hibernate/JPA, covering entities, repositories, service layer, and performance optimization.*

Concept Overview

Layer	Technology	Purpose
Entity	`@Entity`, `@Table`, `@ManyToOne`	Maps Java objects to database tables
Repository	`JpaRepository`, `@Query`	Data access without writing SQL
Service	`@Transactional`, dirty checking	Business logic with automatic transaction management
DTO	Record classes	Decouple API from internal entities

Step 1: Define Entities with Relationships

Concept: Each `@Entity` class maps to a database table. Relationships use `@ManyToOne` (owning side with FK) and `@OneToMany` (inverse side).

```
@Entity
@Table(name = "users",
    indexes = @Index(name = "idx_user_email", columnList = "email"))
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String firstName;

    @Column(nullable = false, unique = true)
    private String email;

    @Enumerated(EnumType.STRING)
```

```
    private UserStatus status = UserStatus.ACTIVE;

    @Embedded                                   // Value object stored in same
table
    private Address address;

    @OneToMany(mappedBy = "user",               // Inverse side -- no FK here
               cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Order> orders = new ArrayList<>();

    @Column(updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    void onCreate() { createdAt = LocalDateTime.now(); }

    // Helper method for bidirectional consistency
    public void addOrder(Order order) {
        orders.add(order);
        order.setUser(this);
    }
}
```

Step 2: Create the Repository

Concept: `JpaRepository` gives you CRUD + paging for free. Custom queries use `@Query` (JPQL) or derived method names.

```
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);          // Derived query --
Spring generates SQL

    boolean existsByEmail(String email);               // EXISTS query --
efficient

    @Query("SELECT u FROM User u WHERE u.status = :status")
    Page<User> findByStatus(@Param("status") UserStatus status, Pageable
pageable);

    @EntityGraph(attributePaths = {"orders"})           // Fetch orders eagerly
(avoids N+1)
```

```
    Optional<User> findWithOrdersById(Long id);

    @Modifying @Transactional
    @Query("UPDATE User u SET u.status = 'INACTIVE' WHERE u.createdAt <
:cutoff")
    int deactivateOldUsers(@Param("cutoff") LocalDateTime cutoff);
}
```

Step 3: Build the Service Layer

Concept: `@Transactional` manages database transactions. Dirty checking automatically saves changes to persistent entities -- no explicit `save()` needed for updates.

```
@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;

    @Transactional
    public UserDTO createUser(CreateUserRequest req) {
        if (userRepository.existsByEmail(req.email())) {
            throw new DuplicateEmailException(req.email());
        }
        User user = User.builder()
            .firstName(req.firstName())
            .email(req.email())
            .build();
        return UserDTO.from(userRepository.save(user));
    }

    @Transactional(readOnly = true)       // Read-only = no dirty checking
overhead
    public Page<UserDTO> listUsers(int page, int size) {
        return userRepository.findAll(PageRequest.of(page, size))
            .map(UserDTO::from);
    }

    @Transactional
    public UserDTO updateUser(Long id, UpdateUserRequest req) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
        user.setFirstName(req.firstName());
```

```
        // NO save() needed -- dirty checking auto-persists on commit
        return UserDTO.from(user);
    }
}
```

Step 4: Performance Optimization

```yaml
spring:
  jpa:
    properties:
      hibernate:
        default_batch_fetch_size: 25     # Batch-fetch lazy collections
        jdbc:
          batch_size: 50                  # Batch INSERTs/UPDATEs
        order_inserts: true               # Group INSERTs by entity type
        order_updates: true
    show-sql: true                        # See generated SQL
```

Key Takeaways

Concept	Remember
`@ManyToOne(fetch = LAZY)`	Always set LAZY -- override EAGER defaults
`mappedBy`	Inverse side has NO FK column -- it's a mirror
Dirty checking	Persistent entities auto-save on transaction commit
`@EntityGraph`	Solve N+1 by fetching associations in a single query
`@Transactional(readOnly = true)`	Better performance for read operations
DTO projections	Don't expose entities directly -- use DTOs

Part 22: Design Patterns Used

22.1 Repository Pattern

What: Encapsulates data access logic behind an interface, providing a collection-like API.

Why: Decouples business logic from data access. Enables testing with mock repositories.

```
// The Repository Pattern is built into Spring Data JPA
public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByCustomerId(Long customerId);
}

// Service depends on interface, not implementation
@Service
public class OrderService {
    private final OrderRepository orderRepository; // Interface!

    public List<Order> getCustomerOrders(Long customerId) {
        return orderRepository.findByCustomerId(customerId);
    }
}

// In tests: use mock
@MockBean
private OrderRepository orderRepository;
```

22.2 DAO Pattern (Data Access Object)

What: Similar to Repository but closer to the persistence mechanism. Often wraps EntityManager directly.

When: When Spring Data JPA's auto-generated methods are insufficient.

```
@Repository
public class CustomOrderDao {

    @PersistenceContext
    private EntityManager em;

    public List<Order> findByComplexCriteria(OrderSearchCriteria criteria) {
```

```
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Order> cq = cb.createQuery(Order.class);
        Root<Order> root = cq.from(Order.class);
        List<Predicate> predicates = new ArrayList<>();
        // Build dynamic query...
        return em.createQuery(cq).getResultList();
    }
}
```

22.3 Specification Pattern

What: Encapsulates query criteria into reusable, composable objects.

When: Dynamic filtering (search pages, dashboards).

```
// Already covered in Part 16 -- Specifications are composable:
Specification<Product> spec = Specification
    .where(hasCategory("Electronics"))
    .and(priceBetween(100, 500))
    .and(nameContains("phone"));
productRepository.findAll(spec, pageable);
```

22.4 Unit of Work Pattern

What: Tracks all changes during a business transaction and persists them as a single unit.

How Hibernate implements it: The Session/EntityManager IS the Unit of Work. It tracks all persistent entities and flushes changes on commit.

```
@Transactional // Defines the Unit of Work boundary
public void processOrder(OrderRequest request) {
    // All changes within this method are tracked
    Order order = new Order(request);
    entityManager.persist(order);      // Tracked

    Customer customer = customerRepo.findById(request.getCustomerId()).get();
    customer.incrementOrderCount();    // Tracked (dirty checking)

    Inventory inv = inventoryRepo.findByProductId(request.getProductId());
    inv.decrementStock(request.getQuantity()); // Tracked
```

```
    // At @Transactional boundary: ALL changes flushed in single transaction
    // INSERT order + UPDATE customer + UPDATE inventory
}
```

Part 23: Best Practices

23.1 Entity Design

```java
// 1. Always override equals() and hashCode() using business key
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;

    @Column(unique = true, nullable = false)
    private String email; // Business key

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof User)) return false;
        User user = (User) o;
        return email != null && email.equals(user.email);
    }

    @Override
    public int hashCode() {
        return Objects.hash(email); // Based on business key, NOT id
    }
}

// 2. Use LAZY fetch by default
@ManyToOne(fetch = FetchType.LAZY) // Override EAGER default!
private Customer customer;

// 3. Use Set instead of List for ManyToMany (avoids duplicates)
@ManyToMany
private Set<Role> roles = new HashSet<>();

// 4. Add helper methods for bidirectional relationships
public void addOrder(Order order) {
    orders.add(order);
    order.setCustomer(this);
}

// 5. Use @PrePersist and @PreUpdate for audit fields
```

```
@PrePersist void onCreate() { createdAt = Instant.now(); }
@PreUpdate void onUpdate() { updatedAt = Instant.now(); }
```

23.2 Transaction Boundaries

```
// 1. Keep transactions SHORT -- only in service layer
@Service
public class UserService {
    @Transactional // HERE -- not in controller or repository
    public void updateUser(Long id, UpdateRequest req) { ... }
}

// 2. Use readOnly for queries (performance boost)
@Transactional(readOnly = true)
public List<UserDTO> searchUsers(SearchCriteria criteria) { ... }

// 3. Avoid calling external APIs inside transactions
@Transactional
public void processPayment(PaymentRequest req) {
    Order order = orderRepo.findById(req.getOrderId()).get();
    // DON'T call external payment API here -- holds DB connection!
}

// BETTER:
public void processPayment(PaymentRequest req) {
    Order order = loadOrder(req.getOrderId()); // @Transactional(readOnly=true)
    PaymentResult result = paymentApi.charge(req); // No transaction
    updateOrderStatus(order.getId(), result); // @Transactional
}
```

23.3 Avoiding LazyInitializationException

```
// Rule: Access all lazy data WITHIN the @Transactional boundary

// Option 1: JOIN FETCH in repository
@Query("SELECT c FROM Customer c JOIN FETCH c.orders WHERE c.id = :id")
Customer findWithOrders(@Param("id") Long id);

// Option 2: @EntityGraph
@EntityGraph(attributePaths = {"orders", "orders.items"})
```

```
Optional<Customer> findById(Long id);

// Option 3: Map to DTO inside transaction
@Transactional(readOnly = true)
public CustomerDTO getCustomer(Long id) {
    Customer c = customerRepo.findById(id).orElseThrow();
    return new CustomerDTO(c.getName(), c.getOrders().size()); // Access here!
}
```

23.4 Query Writing Rules

1. **Never use `SELECT *` in production** -- use DTO projections
2. **Add indexes** for columns used in WHERE, JOIN, ORDER BY
3. **Paginate large results** -- never return unbounded lists
4. **Use @BatchSize** for collections to prevent N+1
5. **Use bulk operations** (@Modifying @Query) for mass updates
6. **Monitor slow queries** with `hibernate.generate_statistics: true`

Part 24: Interview Questions (100+)

Hibernate Fundamentals (Q1-Q25)

Q1. What is ORM and why do we need it? ORM maps Java objects to database tables automatically, eliminating manual JDBC code, ResultSet mapping, and SQL string building. It solves the object-relational impedance mismatch.

Q2. What is the difference between JPA and Hibernate? JPA is a specification (interface/standard). Hibernate is an implementation of JPA. JPA defines annotations like `@Entity`, `@Id`; Hibernate provides the actual code that makes them work.

Q3. What is a SessionFactory? A heavyweight, thread-safe, immutable factory created once at startup. It holds compiled mappings, caches, and connection pool configuration. One SessionFactory per database.

Q4. What is a Session/EntityManager? A lightweight, short-lived, non-thread-safe object that wraps a JDBC connection. It provides the API for CRUD operations and serves as the first-level cache.

Q5. Explain the Hibernate entity lifecycle. Four states: Transient (new, not managed), Persistent (managed by Session, changes auto-tracked), Detached (was persistent, Session closed), Removed (scheduled for deletion).

Q6. What is dirty checking? Hibernate automatically detects changes to persistent entities by comparing current state to a snapshot taken when the entity was loaded. Modified entities generate UPDATE SQL on flush.

Q7. What is the first-level cache? The Session-level cache. Within a Session, the same entity (by ID) is always the same Java object. Second `findById()` returns the cached instance, not a new DB query.

Q8. What is the second-level cache? A SessionFactory-level cache shared across all Sessions. Requires configuration (EhCache, Hazelcast). Caches entity data beyond individual Sessions.

Q9. What is the difference between `persist()` and `save()`? `persist()` is JPA standard -- makes entity persistent, may not return ID immediately. `save()` is Hibernate-specific -- returns the generated ID immediately.

Q10. What is the difference between `merge()` and `update()`? `merge()` copies detached entity state to a new persistent instance (returns the managed copy). `update()` reattaches the detached entity itself (Hibernate-specific, can throw if another instance with same ID is in Session).

Q11. What is `@GeneratedValue` strategy SEQUENCE vs IDENTITY? SEQUENCE uses a database sequence object, allowing batch inserts (Hibernate pre-allocates IDs). IDENTITY uses auto-increment columns but disables batch inserts because Hibernate must execute INSERT to get the ID.

Q12. What is an @Embeddable? A value object that maps to columns in the parent entity's table (no separate table). Example: Address embedded in Customer table.

Q13. What is the difference between @Embedded and @OneToOne? @Embedded stores data in the same table (value object, no identity). @OneToOne uses a separate table with foreign key (has its own identity/lifecycle).

Q14. What is cascade in Hibernate? Cascade propagates operations from parent to child. CascadeType.PERSIST cascades save, REMOVE cascades delete, ALL cascades everything.

Q15. What is orphanRemoval? When a child entity is removed from the parent's collection, it is automatically deleted from the database. Different from CASCADE.REMOVE which only deletes children when the parent is deleted.

Q16. Explain owning side vs inverse side. The owning side has the foreign key column (@JoinColumn). The inverse side uses `mappedBy`. Only changes to the owning side are persisted to the database.

Q17. What is FetchType.LAZY vs EAGER? LAZY loads associated data only when accessed (proxy). EAGER loads immediately via JOIN. LAZY is preferred for performance.

Q18. What is the N+1 problem? Loading N entities and then lazily loading a relationship for each generates 1 + N queries. Solved with JOIN FETCH, @EntityGraph, or @BatchSize.

Q19. What is HQL vs JPQL? HQL is Hibernate-specific query language. JPQL is the JPA standard. Both operate on entities (not tables). JPQL is portable across JPA providers.

Q20. When would you use native SQL? For database-specific features (window functions, CTEs, full-text search), complex queries that are difficult in JPQL, or performance-critical queries.

Q21. What is @NamedQuery? Pre-compiled JPQL queries defined on entity classes. Validated at startup, cached, and reusable. Replaced by @Query in Spring Data JPA.

Q22. What is optimistic vs pessimistic locking? Optimistic: uses @Version column, detects conflicts at commit time. Pessimistic: locks database rows using SELECT FOR UPDATE, prevents concurrent access.

Q23. What is the @Version annotation? Implements optimistic locking. Hibernate checks the version column on UPDATE; if it changed since read, throws OptimisticLockException.

Q24. What is a Hibernate Dialect? Translates HQL/JPQL to database-specific SQL. Different databases have different SQL syntax (LIMIT vs ROWNUM, sequences vs auto-increment).

Q25. What is `spring.jpa.hibernate.ddl-auto`? Controls schema generation: `none` (no action), `validate` (check schema), `update` (alter tables), `create` (drop and create), `create-drop` (create on startup, drop on shutdown). Production should use `validate` or `none`.

Spring Data JPA Questions (Q26-Q50)

Q26. What is Spring Data JPA? An abstraction layer on top of JPA that generates repository implementations from interfaces at runtime, eliminating boilerplate data access code.

Q27. What is the difference between CrudRepository and JpaRepository? JpaRepository extends CrudRepository and adds JPA-specific methods: `flush()`, `saveAndFlush()`, `deleteAllInBatch()`, and supports pagination and sorting.

Q28. How does query method derivation work? Spring Data parses method names and generates JPQL. `findByEmailAndStatus(String email, Status status)` becomes `SELECT u FROM User u WHERE u.email = ?1 AND u.status = ?2`.

Q29. What is the @Query annotation? Defines custom JPQL or native SQL queries on repository methods when method name derivation is insufficient.

Q30. What is @Modifying? Required on @Query methods that modify data (UPDATE, DELETE). Must be used with @Transactional.

Q31. How does pagination work? Pass `Pageable` parameter to repository method. Returns `Page<T>` with content, total count, page info. `PageRequest.of(page, size, sort)` creates Pageable.

Q32. What is a Specification? A reusable query predicate. Implements `Specification<T>` interface. Composable with `.and()`, `.or()`. Requires `JpaSpecificationExecutor`.

Q33. What is an @EntityGraph? Defines which associations to fetch eagerly for a specific query, overriding default fetch types. Prevents N+1 without JOIN FETCH in query string.

Q34. What is a DTO projection? Selecting only specific columns into a DTO instead of full entity. Reduces data transfer and avoids loading unnecessary relationships.

Q35. What is the difference between interface and class projections? Interface projections: Spring generates proxy implementing getter methods. Class projections: constructor-based, JPQL `SELECT NEW`.

Q36. How to handle soft deletes? Use `@SQLDelete` and `@Where` annotations or manually add `deletedAt` column with global filter.

Q37. What is Auditing in Spring Data JPA? Auto-populating `createdBy`, `createdDate`, `lastModifiedBy`, `lastModifiedDate` using `@EnableJpaAuditing` and `@CreatedDate`, `@LastModifiedDate` annotations.

Q38. What is @Transactional(readOnly = true)? Tells Hibernate to skip dirty checking for entities loaded in this transaction, improving performance. May also route to read replicas.

Q39. What happens if @Transactional is on a private method? It does NOT work. Spring AOP creates proxies for public methods only. @Transactional on private methods is silently ignored.

Q40. What is the Open Session in View pattern? Keeps the Hibernate Session open until the view is rendered (controller returns response). Prevents LazyInitializationException but can cause N+1 queries in views. `spring.jpa.open-in-view=false` to disable (recommended).

Q41. How to implement custom repository methods? Create a custom interface + implementation class. Spring Data merges it with the auto-generated implementation.

Q42. What is QueryByExample? Creates queries from entity instances. `productRepository.findAll(Example.of(probe))` where probe is a partially filled entity.

Q43. How to use multiple data sources? Configure separate DataSource, EntityManagerFactory, and TransactionManager beans. Use `@EnableJpaRepositories(basePackages, entityManagerFactoryRef)` per datasource.

Q44. What is @Param in @Query? Binds method parameters to named query parameters: `@Query("... WHERE u.email = :email") User findByEmail(@Param("email") String email)`.

Q45. What is saveAndFlush() vs save()? `save()` persists but may delay SQL until transaction commit. `saveAndFlush()` immediately executes SQL, useful when you need the generated ID right away.

Q46. How to call stored procedures? Use `@Procedure` annotation or `@NamedStoredProcedureQuery` on the entity.

Q47. What is @Lock in Spring Data JPA? Applies pessimistic or optimistic lock mode to a query: `@Lock(LockModeType.PESSIMISTIC_WRITE)`.

Q48. How to handle database migrations? Use Flyway or Liquibase. Never use `ddl-auto: update` in production. Version all schema changes as migration scripts.

Q49. What is the difference between findById and getById? `findById()` returns `Optional<T>`, immediately executes SELECT. `getReferenceById()` returns a lazy proxy, throws EntityNotFoundException on access if not found.

Q50. How to test repositories? Use `@DataJpaTest` with embedded H2. It auto-configures an in-memory database, scans for entities and repositories, wraps tests in transactions that rollback.

Performance and Architecture Questions (Q51-Q75)

Q51. How to solve the N+1 problem? Four solutions: JOIN FETCH in JPQL, @EntityGraph, @BatchSize on collections, DTO projections that avoid relationships entirely.

Q52. How to optimize batch inserts? Use SEQUENCE strategy (not IDENTITY), set `hibernate.jdbc.batch_size=50`, enable `order_inserts=true`, flush and clear EntityManager periodically.

Q53. What is connection pool sizing best practice? Formula: connections = (core_count * 2) + disk_spindles. For most apps, 10-20 connections with HikariCP.

Q54. How to detect slow queries? Enable `hibernate.generate_statistics=true` and `show_sql=true`. Use slow query logging in the database. Monitor with APM tools.

Q55. What is the difference between JPQL JOIN and JOIN FETCH? JOIN filters results but doesn't load the association. JOIN FETCH loads the association into the entity (initializes the lazy proxy).

Q56. When to use native queries? Database-specific functions, window functions, CTEs, full-text search, performance-critical queries, complex aggregations.

Q57. How to handle large result sets? Use pagination (Pageable), streaming (Stream), or ScrollableResults. Never load millions of entities into memory.

Q58. What is the impact of CascadeType.ALL? Cascades all operations including REMOVE. Can cause unexpected deletions. Use specific cascades (PERSIST, MERGE) unless parent truly owns child lifecycle.

Q59. How to prevent LazyInitializationException? Fetch data within @Transactional, use JOIN FETCH, use @EntityGraph, map to DTO inside service layer, or disable Open Session in View.

Q60. What is database indexing strategy for JPA? Index columns used in WHERE, JOIN, and ORDER BY clauses. Use `@Index` on `@Table` or manage via Flyway/Liquibase migrations.

Q61. How to implement multi-tenancy? Database per tenant (separate datasources), Schema per tenant (schema switching), or Discriminator column (shared table with tenant_id filter).

Q62. What is OSIV (Open Session in View) and why disable it? Keeps Session open through controller response rendering. Causes lazy queries in view layer (N+1 risk). Disable with `spring.jpa.open-in-view=false`.

Q63. How to handle concurrent updates? Use @Version for optimistic locking. Handle OptimisticLockException with retry logic in service layer.

Q64. What is the difference between first and second level cache? L1: per Session, automatic, cannot be disabled. L2: per SessionFactory, shared, configurable, requires explicit opt-in per entity.

Q65. How to monitor Hibernate in production? Enable statistics, export metrics to Prometheus/Grafana (via Micrometer), log slow queries, monitor connection pool metrics (HikariCP).

Q66. What is @DynamicUpdate? Generates UPDATE SQL with only changed columns, not all columns. Useful for tables with many columns.

Q67. What is @NaturalId? Marks a business key (like email, SSN). Hibernate provides a cache-friendly lookup by natural ID.

Q68. How to handle inheritance in JPA? Three strategies: SINGLE_TABLE (one table, discriminator column), JOINED (table per class with JOIN), TABLE_PER_CLASS (separate tables, no JOIN).

Q69. When to use SINGLE_TABLE vs JOINED inheritance? SINGLE_TABLE: best performance (no JOINs), nullable columns. JOINED: normalized, no nulls, but slower due to JOINs.

Q70. What is @MappedSuperclass vs @Inheritance? @MappedSuperclass: shares fields but no polymorphic queries. @Inheritance: full polymorphism with `FROM BaseEntity` queries.

Q71. How to implement soft delete with JPA? Add `deletedAt` column, override delete with `@SQLDelete(sql = "UPDATE ... SET deleted_at = NOW()")`, filter with `@Where(clause = "deleted_at IS NULL")`.

Q72. What is EntityManager.flush()? Synchronizes persistence context with database (executes pending SQL) WITHOUT committing the transaction. Useful before native queries.

Q73. What is the difference between em.persist() and repo.save()? `persist()` is JPA standard, only works on new entities. `save()` calls `persist()` for new or `merge()` for existing (based on ID presence).

Q74. How to handle time zones in JPA? Store as `Instant` or `OffsetDateTime` in UTC. Configure JVM timezone: `-Duser.timezone=UTC`. Use `@Column(columnDefinition =`

`"TIMESTAMP WITH TIME ZONE")`.

Q75. What is the best entity equals/hashCode strategy? Use business key (email, ISBN), never the generated ID (changes from null to value on persist). If no business key, use UUID set in constructor.

Architect-Level Questions (Q76-Q100)

Q76. Design a multi-tenant SaaS persistence layer. Use discriminator-based tenancy with `tenant_id` column. Implement Hibernate filter to automatically add `WHERE tenant_id = ?`. Use `ThreadLocal` to store current tenant.

Q77. How to handle database migrations in CI/CD? Flyway migration scripts versioned in Git. CI pipeline runs `flyway:migrate` before deployment. Rollback scripts for each migration. Never use `ddl-auto`.

Q78. Design read/write splitting with JPA. Configure two datasources (primary for writes, replica for reads). Use `@Transactional(readOnly = true)` to route to replica. Implement `AbstractRoutingDataSource`.

Q79. How to implement CQRS with JPA? Write model: full entities with JPA for commands. Read model: DTO projections or separate read-optimized views. Optionally synchronize via events.

Q80. Design an audit logging system with JPA. Use Hibernate Envers or custom `@EntityListeners` with `@PrePersist`, `@PreUpdate`, `@PreRemove`. Store audit trail in separate audit tables.

Q81. How to handle schema evolution in microservices? Each service owns its schema. Use backward-compatible migrations (add columns, never remove). Blue-green deployment with migration windows.

Q82. Design a high-performance bulk import system. Stateless Session, JDBC batch inserts, disable auto-commit, clear persistence context every N rows, use database COPY command for massive imports.

Q83. How to implement event sourcing with JPA? Store events as entities in an events table. Rebuild state by replaying events. Use `@DomainEvents` in Spring Data for publishing.

Q84. How to handle distributed caching with Hibernate L2? Use Hazelcast or Redis as L2 cache provider. Configure cache regions per entity. Handle cache invalidation across cluster nodes.

Q85. Design a full-text search with JPA. Use Hibernate Search (Lucene/Elasticsearch integration) or native database full-text indexes with native queries.

Q86. How to profile and optimize a slow JPA application? Enable Hibernate statistics, identify N+1 queries, check for missing indexes, analyze query plans (EXPLAIN), add DTO projections, configure batch

fetching.

Q87. Design a versioned API with different entity shapes. Use DTO projection per API version. Keep entity stable; vary DTOs. Version endpoints: `/api/v1/users`, `/api/v2/users`.

Q88. How to handle eventual consistency between services? Outbox pattern: write event to outbox table in same transaction, background process publishes to Kafka. Consumer is idempotent.

Q89. When would you NOT use an ORM? Extreme performance requirements, complex reporting (use raw SQL/views), simple CRUD with few entities (use Spring JDBC), NoSQL databases.

Q90. Design a data archival strategy with JPA. Partition tables by date. Batch job moves old data to archive tables. Use separate entity mappings for archive. Maintain foreign key integrity.

Q91. How to handle database connection issues gracefully? HikariCP connection validation, circuit breaker pattern (Resilience4j), retry with exponential backoff, health checks for database connectivity.

Q92. Design the persistence layer for an e-commerce platform. User, Product, Order, OrderItem, Payment, Address entities. Aggregate root pattern (Order is aggregate root for OrderItems). Optimistic locking on inventory. Event-driven stock updates.

Q93. How to implement row-level security with JPA? Hibernate `@Filter` with `@FilterDef`. Enable filter per Session with tenant/user context. Or use database-level row-level security (PostgreSQL RLS).

Q94. What is the Outbox Pattern? Write domain event to an outbox table within the same transaction as the business operation. A separate process reads the outbox and publishes to message broker. Guarantees at-least-once delivery.

Q95. How to handle large BLOB/CLOB data? Use `@Lob` with lazy loading. Store large files in object storage (S3), save only reference in DB. Use streaming for large reads.

Q96. Design a polymorphic search across entity types. Use SINGLE_TABLE inheritance with discriminator. Query base entity with Specifications. Or use Elasticsearch for cross-entity search.

Q97. How to implement database sharding with JPA? Custom `AbstractRoutingDataSource` that routes based on shard key. Each shard has own EntityManagerFactory. Application manages shard selection.

Q98. What is the Unit of Work pattern in Hibernate? The Session IS the Unit of Work. It tracks all entity changes (inserts, updates, deletes) and flushes them as a batch in a single transaction on commit.

Q99. How to implement the Specification pattern for complex filters? Create reusable Specification classes. Compose with `.and()`, `.or()`. Support dynamic filter UI. Test each specification independently.

Q100. Design a reporting system that doesn't impact OLTP performance. Use read replica for reports. Materialize views for complex aggregations. Use Spring Batch for scheduled report generation. DTO projections for lightweight queries.

Q101. What is the difference between Spring Data JPA and Spring Data JDBC? JPA: full ORM, lazy loading, caching, entity lifecycle. JDBC: no ORM, no lazy loading, no session cache. JDBC is simpler but less powerful.

Q102. How to handle database transactions across multiple repositories? One @Transactional on the service method wraps all repository calls in a single transaction. Don't put @Transactional on repositories.

Q103. What is the N+1 problem in aggregations? Loading aggregate data by iterating entities instead of using GROUP BY queries. Solution: use @Query with aggregation functions or native SQL.

Part 25: Coding Exercises (20)

Exercise 1: Basic Entity with Auditing

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class Article {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(columnDefinition = "TEXT")
    private String content;

    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy
    @Column(updatable = false)
    private String createdBy;
}
```

Exercise 2: Composite Key Entity

```
@Embeddable
public class EnrollmentId implements Serializable {
    private Long studentId;
    private Long courseId;
    // equals() and hashCode()
}

@Entity
public class Enrollment {
```

```
    @EmbeddedId
    private EnrollmentId id;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("studentId")
    private Student student;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("courseId")
    private Course course;

    private LocalDate enrolledDate;
    private String grade;
}
```

Exercise 3: OneToMany with Cascade and OrphanRemoval

```
@Entity
public class ShoppingCart {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy = "cart", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<CartItem> items = new ArrayList<>();

    public void addItem(Product product, int quantity) {
        CartItem item = new CartItem(this, product, quantity);
        items.add(item);
    }

    public void removeItem(CartItem item) {
        items.remove(item);
        item.setCart(null);
    }
}
```

Exercise 4: Inheritance Mapping (SINGLE_TABLE)

```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "payment_type")
public abstract class Payment {
    @Id @GeneratedValue private Long id;
    private BigDecimal amount;
    private LocalDateTime paidAt;
}

@Entity
@DiscriminatorValue("CREDIT_CARD")
public class CreditCardPayment extends Payment {
    private String cardNumber;
    private String expiryDate;
}

@Entity
@DiscriminatorValue("BANK_TRANSFER")
public class BankTransferPayment extends Payment {
    private String bankCode;
    private String accountNumber;
}
```

Exercise 5: Query Method Derivation

```java
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByDepartmentAndStatusOrderBySalaryDesc(
        String department, EmployeeStatus status);

    Optional<Employee> findFirstByDepartmentOrderBySalaryDesc(
        String department); // Highest paid in department

    long countByDepartment(String department);
    boolean existsByEmail(String email);
    List<Employee> findTop5ByOrderBySalaryDesc(); // Top 5 earners
}
```

Exercise 6: Custom @Query with DTO Projection

```java
public record DepartmentStats(
    String department, long employeeCount, BigDecimal avgSalary) {}

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    @Query("SELECT new com.app.dto.DepartmentStats(" +
           "e.department, COUNT(e), AVG(e.salary)) " +
           "FROM Employee e GROUP BY e.department " +
           "ORDER BY AVG(e.salary) DESC")
    List<DepartmentStats> getDepartmentStatistics();
}
```

Exercise 7: Pagination with Specification

```java
@Service
public class EmployeeSearchService {

    @Autowired private EmployeeRepository repo;

    public Page<Employee> search(String name, String dept,
                                 BigDecimal minSalary, Pageable pageable) {
        Specification<Employee> spec = Specification.where(null);
        if (name != null) spec = spec.and(nameContains(name));
        if (dept != null) spec = spec.and(inDepartment(dept));
        if (minSalary != null) spec = spec.and(salaryGreaterThan(minSalary));
        return repo.findAll(spec, pageable);
    }

    private Specification<Employee> nameContains(String name) {
        return (r, q, cb) -> cb.like(cb.lower(r.get("name")),
            "%" + name.toLowerCase() + "%");
    }

    private Specification<Employee> inDepartment(String dept) {
        return (r, q, cb) -> cb.equal(r.get("department"), dept);
    }

    private Specification<Employee> salaryGreaterThan(BigDecimal min) {
        return (r, q, cb) -> cb.greaterThanOrEqualTo(r.get("salary"), min);
    }
}
```

Exercise 8: Optimistic Locking

```java
@Entity
public class InventoryItem {
    @Id private Long id;
    private String productName;
    private int quantity;

    @Version
    private Long version; // Optimistic lock column

    public void decrementStock(int amount) {
        if (quantity < amount)
            throw new InsufficientStockException();
        this.quantity -= amount;
    }
}

@Service
public class InventoryService {
    @Transactional
    @Retryable(value = OptimisticLockException.class, maxAttempts = 3)
    public void reserveStock(Long itemId, int amount) {
        InventoryItem item = repo.findById(itemId).orElseThrow();
        item.decrementStock(amount);
        // If another thread updated concurrently, version mismatch
        // throws OptimisticLockException -> retry
    }
}
```

Exercise 9: Entity Lifecycle Callbacks

```java
@Entity
public class Order {
    @Id @GeneratedValue private Long id;
    private BigDecimal amount;
    private String status;

    @PrePersist  void prePersist()  { log.info("About to INSERT order"); }
    @PostPersist void postPersist() { log.info("INSERTed order id={}", id); }
    @PreUpdate   void preUpdate()   { log.info("About to UPDATE order {}", id);
```

```
}
    @PostUpdate  void postUpdate()  { log.info("UPDATEd order {}", id); }
    @PreRemove   void preRemove()   { log.info("About to DELETE order {}", id);
}
    @PostRemove  void postRemove()  { log.info("DELETEd order {}", id); }
    @PostLoad    void postLoad()    { log.info("LOADed order {}", id); }
}
```

Exercise 10: Bulk Update with @Modifying

```
public interface UserRepository extends JpaRepository<User, Long> {
    @Modifying(clearAutomatically = true)
    @Transactional
    @Query("UPDATE User u SET u.status = 'INACTIVE' " +
           "WHERE u.lastLoginAt < :cutoff")
    int deactivateInactiveUsers(@Param("cutoff") LocalDateTime cutoff);
}
```

Exercise 11: ManyToMany with Extra Columns

```
// When join table needs extra columns, use an entity instead of @ManyToMany
@Entity
public class ProjectAssignment {
    @EmbeddedId
    private ProjectAssignmentId id;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("employeeId")
    private Employee employee;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("projectId")
    private Project project;

    private String role;          // Extra column
    private LocalDate assignedDate; // Extra column
}
```

Exercise 12: Custom Repository Implementation

```java
public interface UserRepositoryCustom {
    List<User> findUsersWithComplexCriteria(UserSearchCriteria criteria);
}

@Repository
public class UserRepositoryCustomImpl implements UserRepositoryCustom {
    @PersistenceContext
    private EntityManager em;

    @Override
    public List<User> findUsersWithComplexCriteria(UserSearchCriteria c) {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<User> query = cb.createQuery(User.class);
        Root<User> root = query.from(User.class);

        List<Predicate> predicates = new ArrayList<>();
        if (c.getName() != null)
            predicates.add(cb.like(root.get("name"), "%" + c.getName() + "%"));
        if (c.getMinAge() != null)
            predicates.add(cb.ge(root.get("age"), c.getMinAge()));

        query.where(predicates.toArray(new Predicate[0]));
        return em.createQuery(query).getResultList();
    }
}

// Combine with Spring Data
public interface UserRepository extends JpaRepository<User, Long>,
                                        UserRepositoryCustom { }
```

Exercise 13: Interface-Based Projection

```java
public interface UserSummary {
    Long getId();
    String getFullName();
    String getEmail();

    @Value("#{target.firstName + ' ' + target.lastName}")
    String getDisplayName(); // Computed from entity fields
```

```
}

public interface UserRepository extends JpaRepository<User, Long> {
    List<UserSummary> findByStatus(UserStatus status);
}
```

Exercise 14: Soft Delete

```
@Entity
@SQLDelete(sql = "UPDATE products SET deleted_at = NOW() WHERE id = ?")
@Where(clause = "deleted_at IS NULL")
public class Product {
    @Id @GeneratedValue private Long id;
    private String name;
    private LocalDateTime deletedAt;
}
// productRepository.deleteById(1L);
// Executes: UPDATE products SET deleted_at = NOW() WHERE id = 1
// productRepository.findAll();
// Executes: SELECT * FROM products WHERE deleted_at IS NULL
```

Exercise 15: EntityGraph for Selective Fetching

```
@Entity
@NamedEntityGraph(name = "Customer.withOrdersAndItems",
    attributeNodes = {
        @NamedAttributeNode(value = "orders",
            subgraph = "orders-items")
    },
    subgraphs = {
        @NamedSubgraph(name = "orders-items",
            attributeNodes = @NamedAttributeNode("items"))
    })
public class Customer { ... }

public interface CustomerRepository extends JpaRepository<Customer, Long> {
    @EntityGraph("Customer.withOrdersAndItems")
```

```
    Optional<Customer> findById(Long id);
}
```

Exercise 16: Stored Procedure Call

```
public interface ReportRepository extends JpaRepository<Report, Long> {

    @Procedure(name = "generate_monthly_report")
    void generateMonthlyReport(@Param("month") int month,
                               @Param("year") int year);
}
```

Exercise 17: Multiple DataSources

```
@Configuration
@EnableJpaRepositories(
    basePackages = "com.app.primary.repository",
    entityManagerFactoryRef = "primaryEntityManagerFactory",
    transactionManagerRef = "primaryTransactionManager")
public class PrimaryDataSourceConfig {

    @Primary
    @Bean
    @ConfigurationProperties("spring.datasource.primary")
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create().build();
    }
}
```

Exercise 18: Query by Example

```
@Transactional(readOnly = true)
public List<Product> searchByExample(String name, String category) {
    Product probe = new Product();
    probe.setName(name);
    probe.setCategory(category);
```

```
    ExampleMatcher matcher = ExampleMatcher.matching()
        .withIgnoreCase()
        .withStringMatcher(ExampleMatcher.StringMatcher.CONTAINING)
        .withIgnoreNullValues();

    return productRepository.findAll(Example.of(probe, matcher));
}
```

Exercise 19: Batch Processing with JPA

```
@Transactional
public void batchInsert(List<Employee> employees) {
    for (int i = 0; i < employees.size(); i++) {
        entityManager.persist(employees.get(i));
        if (i % 50 == 0) { // Flush every 50
            entityManager.flush();
            entityManager.clear(); // Free memory
        }
    }
}
```

Exercise 20: Testing Repository with @DataJpaTest

```
@DataJpaTest
class UserRepositoryTest {

    @Autowired private UserRepository userRepository;
    @Autowired private TestEntityManager entityManager;

    @Test
    void shouldFindByEmail() {
        User user = new User("John", "john@test.com");
        entityManager.persistAndFlush(user);

        Optional<User> found = userRepository.findByEmail("john@test.com");

        assertThat(found).isPresent();
        assertThat(found.get().getName()).isEqualTo("John");
    }
```

```java
    @Test
    void shouldReturnEmptyForNonExistentEmail() {
        Optional<User> found = userRepository.findByEmail("x@test.com");
        assertThat(found).isEmpty();
    }

    @Test
    void shouldPaginateResults() {
        for (int i = 0; i < 25; i++) {
            entityManager.persist(new User("User" + i, "u" + i + "@test.com"));
        }
        entityManager.flush();

        Page<User> page = userRepository.findAll(PageRequest.of(0, 10));

        assertThat(page.getContent()).hasSize(10);
        assertThat(page.getTotalElements()).isEqualTo(25);
        assertThat(page.getTotalPages()).isEqualTo(3);
    }
}
```

End of Guide

Document Version: 1.0 **Last Updated:** March 2026 **Topics Covered:** 25 Parts, 103 Interview Questions, 20 Coding Exercises